

TIỀN XỬ LÝ, REGULARIZATION & CHỌN ĐẶC TRƯNG

Data Preprocessing · Regularization · Feature Selection

TS. Cao Tiến Dũng



**FPT SCHOOL OF BUSINESS
& TECHNOLOGY**

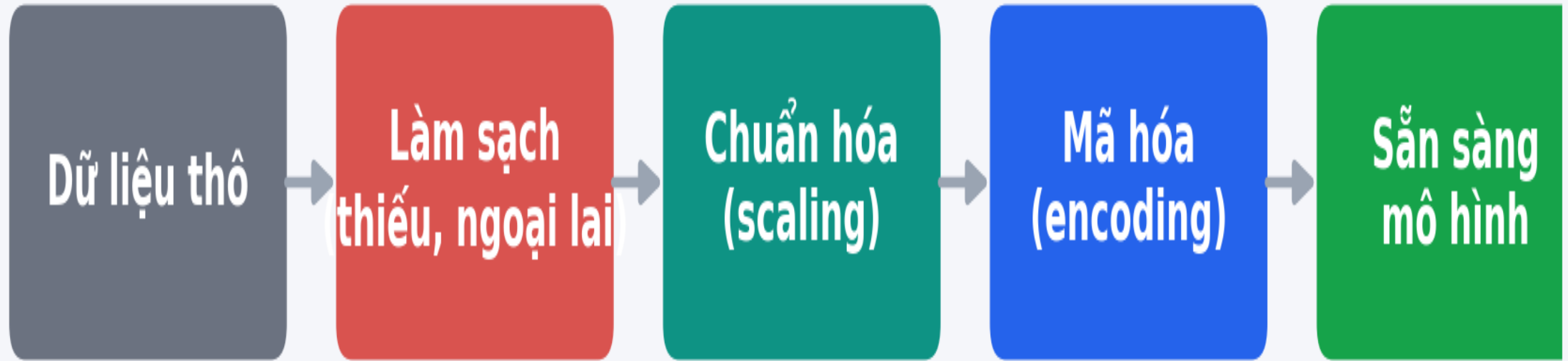


01

Xử lý dữ liệu

Data Preprocessing

Tổng quan quy trình tiền xử lý



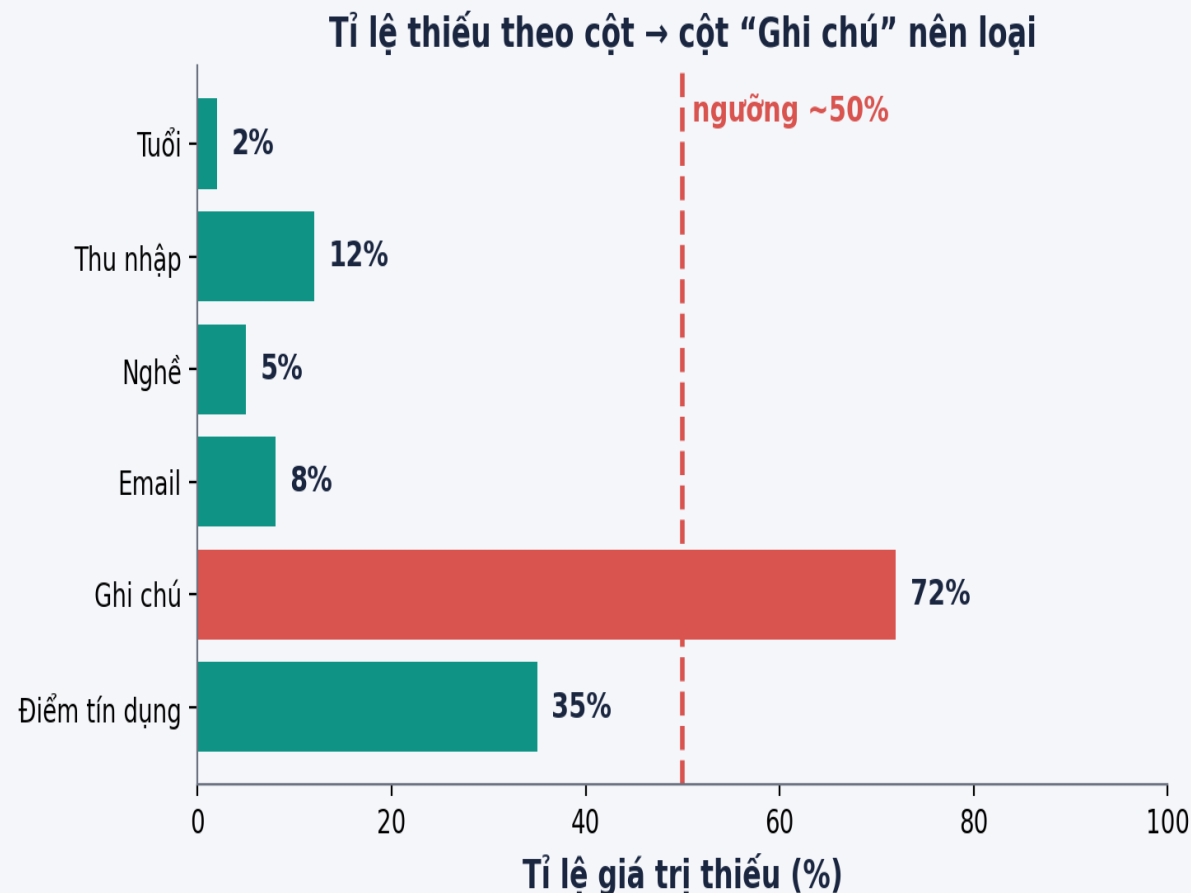
- **“Garbage in, garbage out”** — chất lượng dữ liệu quyết định chất lượng mô hình; tiền xử lý thường chiếm phần lớn thời gian của một dự án ML.
- **Bốn nhóm việc chính:** làm sạch (thiếu, ngoại lai) → biến đổi (chuẩn hóa, mã hóa) → đưa dữ liệu về dạng mô hình học được.

Dữ liệu thiếu: nhận diện & phân loại

- Ba cơ chế thiếu dữ liệu:

- **MCAR** — thiếu ngẫu nhiên hoàn toàn (độc lập mọi biến).
- **MAR** — thiếu phụ thuộc các biến KHÁC đã quan sát.
- **MNAR** — thiếu phụ thuộc chính giá trị bị thiếu
→ “bị thiếu” mang thông tin.

- **Phát hiện:** `df.isnull().sum()`, tỉ lệ thiếu mỗi cột, heatmap thiếu.



Xử lý dữ liệu thiếu: chiến lược

- **Xóa:** bỏ cột nếu thiếu quá ngưỡng (vd >50–60%); bỏ hàng nếu thiếu ít, ngẫu nhiên.
- **Điền khuyết (impute):** mean/median/mode, KNN, Iterative (MICE).
- **Cờ is_missing:** khi MNAR — giữ tín hiệu “bị thiếu” thay vì xóa.

```
from sklearn.impute import \
    SimpleImputer, KNNImputer

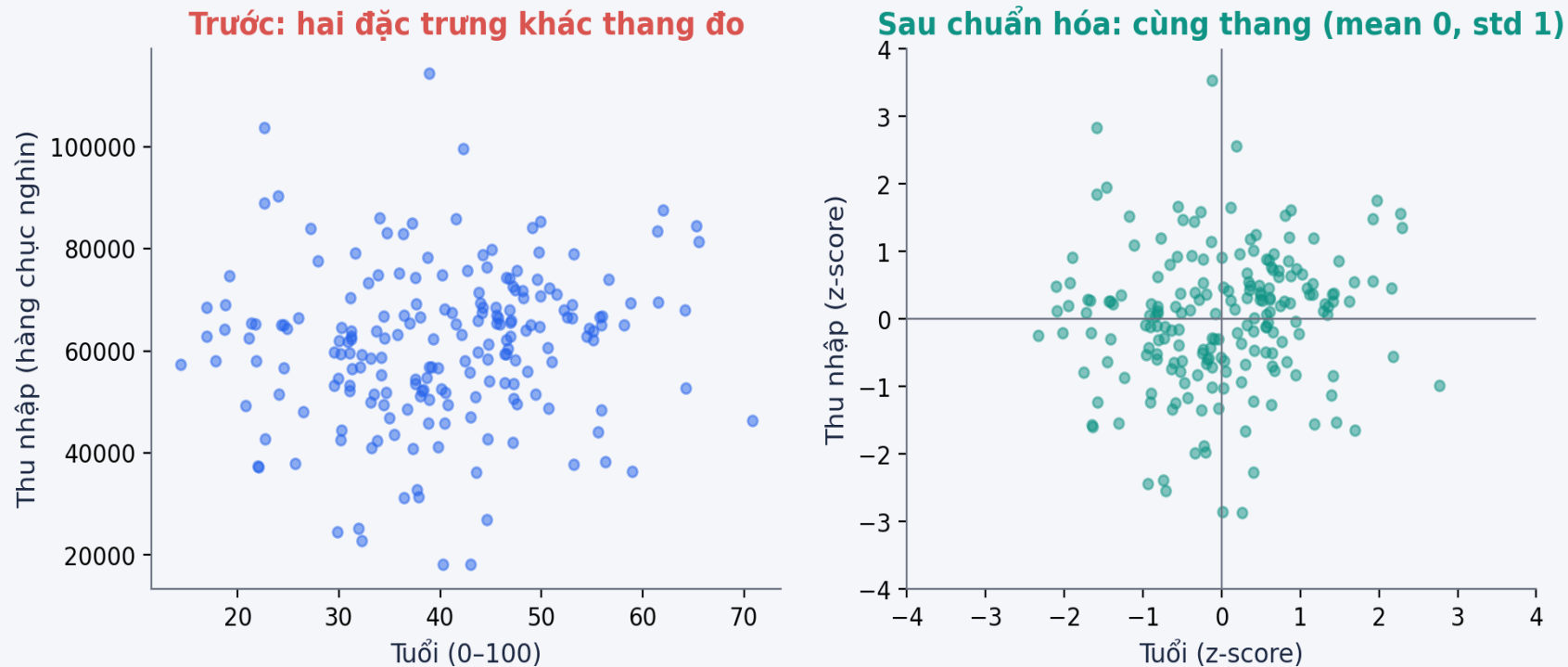
# Cột số → điền trung vị
num = SimpleImputer(strategy="median")
X_num = num.fit_transform(X_num)

# Cột phân loại → giá trị hay gặp nhất
cat = SimpleImputer(strategy="most_frequent")

# Hoặc KNN theo mẫu lân cận
X =
KNNImputer(n_neighbors=5).fit_transform(X)

# Cờ thiếu (MNAR)
df["age_NA"] = df["age"].isnull().astype(int)
```

Vì sao cần chuẩn hóa (Feature Scaling)?



- **Vấn đề:** đặc trưng khác thang đo (Tuổi 0–100 vs Thu nhập hàng triệu) khiến biến lớn “lấn át” khi tính khoảng cách/gradient.
- **Bị ảnh hưởng:** kNN, SVM, Gradient Descent, Regularization, PCA. (Cây quyết định/Random Forest thì KHÔNG cần.)

Standardization vs Min-Max vs Robust

Standardization

$$z = \frac{x - \mu}{\sigma}$$

Min-Max

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

Robust (median/IQR)

Phương pháp	Kết quả	Bền ngoại lai?	Khi nào dùng
Standardization	mean 0, std 1	Khá	Mặc định (đa số mô hình tuyến tính, SVM, NN)
Min-Max	ép về [0, 1]	Kém	Cần biên rõ ràng (ảnh, mạng nơ-ron)
Robust	theo median & IQR	Tốt	Dữ liệu nhiều điểm ngoại lai

Mã hóa biến phân loại (Categorical Encoding)

Phương pháp	Ý tưởng	Phù hợp	Lưu ý
One-Hot	Mỗi hạng mục $\rightarrow$ 1 cột nhị phân	Biến danh nghĩa (không thứ tự)	Tăng số chiều nếu nhiều hạng mục
Label / Ordinal	Gán số nguyên theo thứ tự	Biến CÓ thứ tự (Nhỏ < Vừa < Lớn)	Đừng dùng cho biến danh nghĩa
Target / Mean	Thay bằng trung bình y của hạng mục	Hạng mục nhiều giá trị (high-cardinality)	Dễ RÒ RỈ dữ liệu $\rightarrow$ cần cross-validation

Ví dụ One-Hot: Màu $\in \{\text{Đỏ, Xanh, Vàng}\} \rightarrow \text{is_Đỏ, is_Xanh, is_Vàng}$ (mỗi hàng đúng một cột = 1).

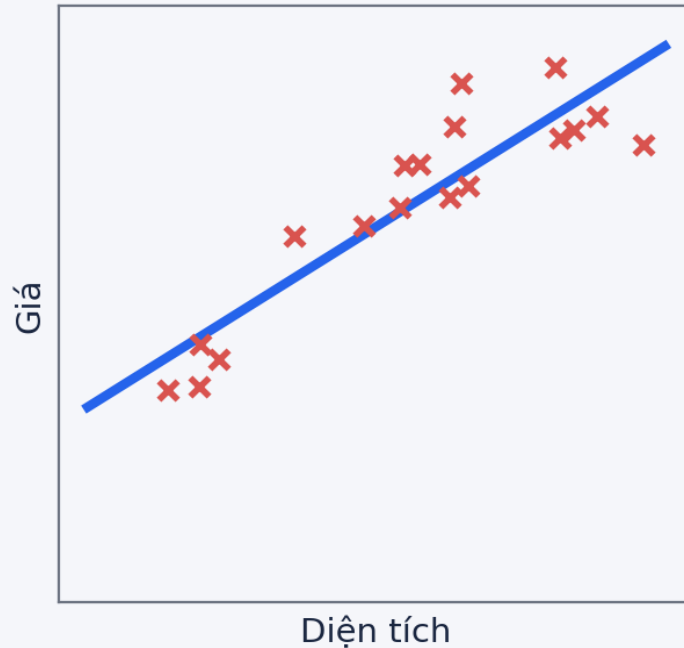
02

Overfitting & hai hướng giải quyết

Overfitting & Two Remedies

Overfitting là gì?

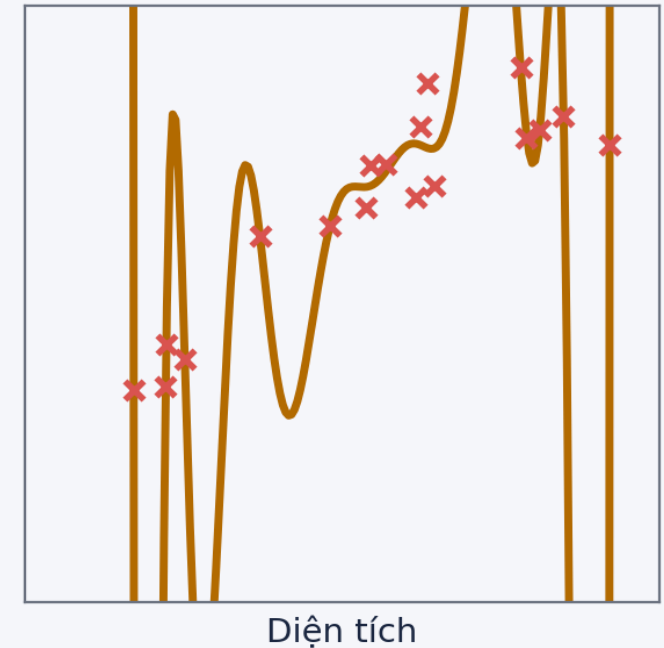
Dưới khớp (High bias)



Vừa phải (Just right)



Quá khớp (High variance)

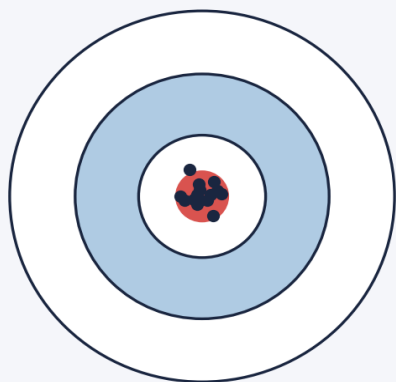


$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 \approx 0$$

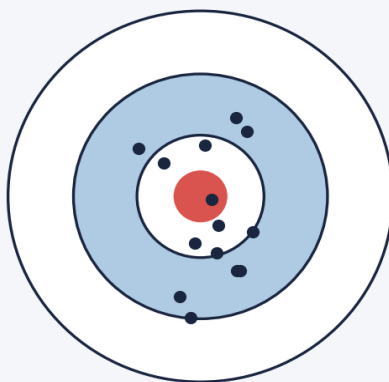
Mô hình quá phức tạp khớp tập huấn luyện gần như hoàn hảo ($J(\theta) \approx 0$) nhưng **tổng quát hóa kém** trên dữ liệu mới.

Bias – Variance

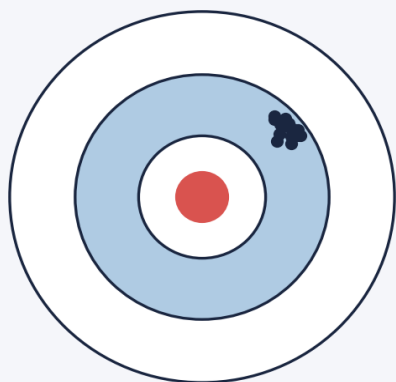
Bias thấp · Variance thấp



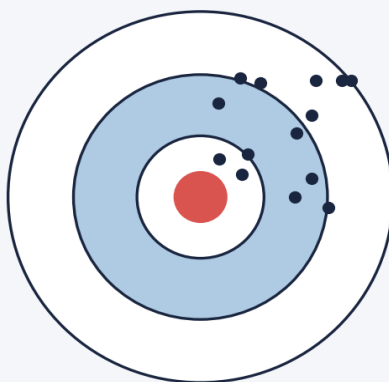
Bias thấp · Variance cao



Bias cao · Variance thấp



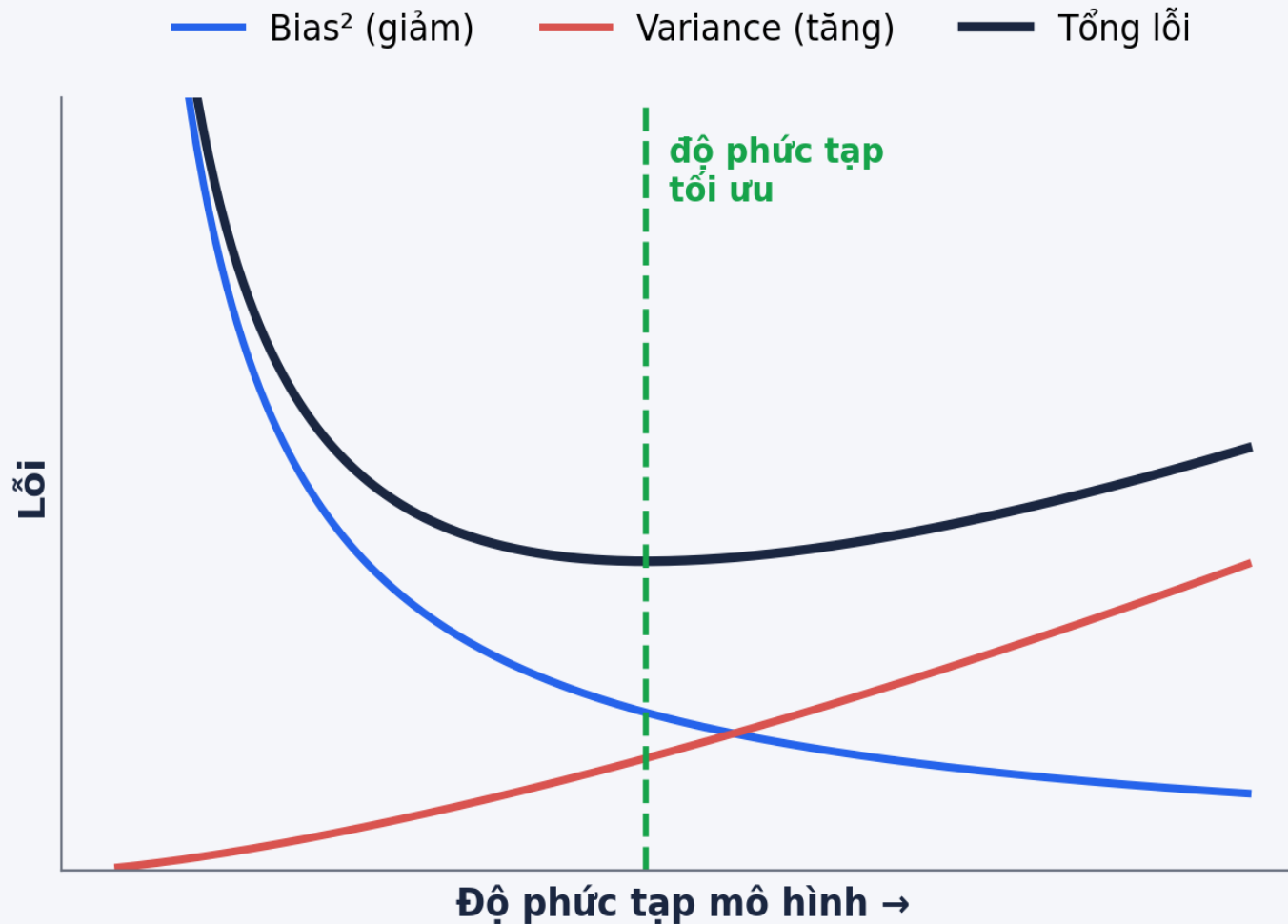
Bias cao · Variance cao



- **Bias (độ chệch):** chênh giữa kỳ vọng học được và sự thật — GIẢM khi mô hình phức tạp hơn.
- **Variance (phương sai):** độ nhạy với tập dữ liệu cụ thể — TĂNG khi mô hình phức tạp hơn.
- **Dưới khớp** = bias cao; **quá khớp** = variance cao.

$$E[(y - \hat{f})^2] = \underbrace{\text{Bias}[\hat{f}]^2} + \underbrace{\text{Var}[\hat{f}]} + \sigma^2$$

Đánh đổi Bias – Variance



- Khi tăng độ phức tạp:
 - Bias² giảm dần.
 - Variance tăng dần.
- Tổng lỗi có dạng chữ U → tồn tại độ phức tạp “vừa phải” tối ưu.
- Mục tiêu: cân bằng, không quá đơn giản cũng không quá phức tạp.

Hai hướng chống Overfitting

OVERFITTING

```
graph TD; A[OVERFITTING] --> B[1 · Regularization]; A --> C[2 · Chọn đặc trưng];
```

1 · Regularization

Giữ MỌI đặc trưng,
thu nhỏ hệ số θ_j

1 · Regularization

- Giữ TẤT CẢ đặc trưng, thu nhỏ độ lớn hệ số θ_j . Hợp khi nhiều đặc trưng, mỗi cái đóng góp một chút.

2 · Chọn đặc trưng

GIẢM số đặc trưng
(bỏ biến nhiễu/trùng)

2 · Chọn đặc trưng

- GIẢM số đặc trưng: bỏ biến nhiễu, trùng lặp, ít liên quan đến mục tiêu y.

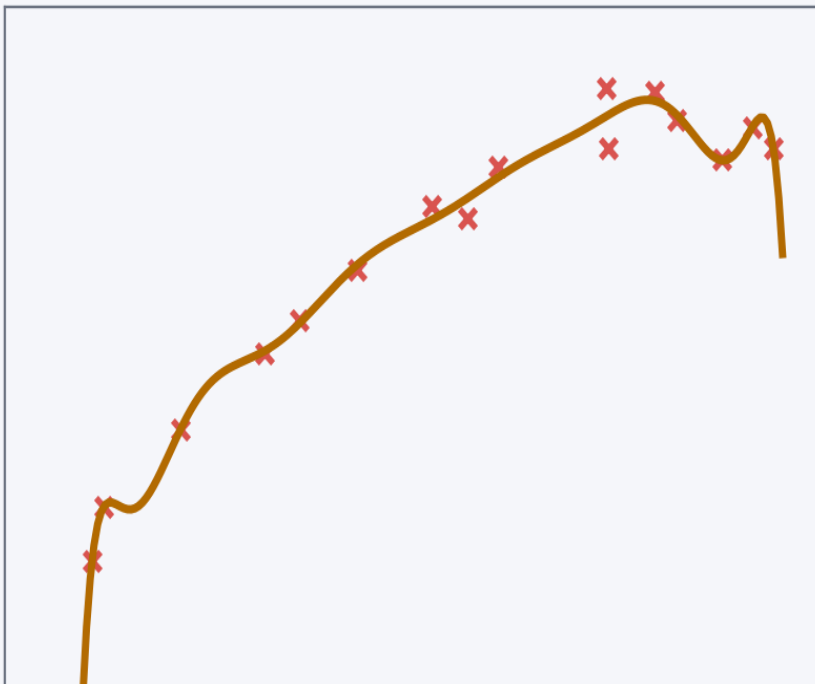
03

Hướng 1: Regularization

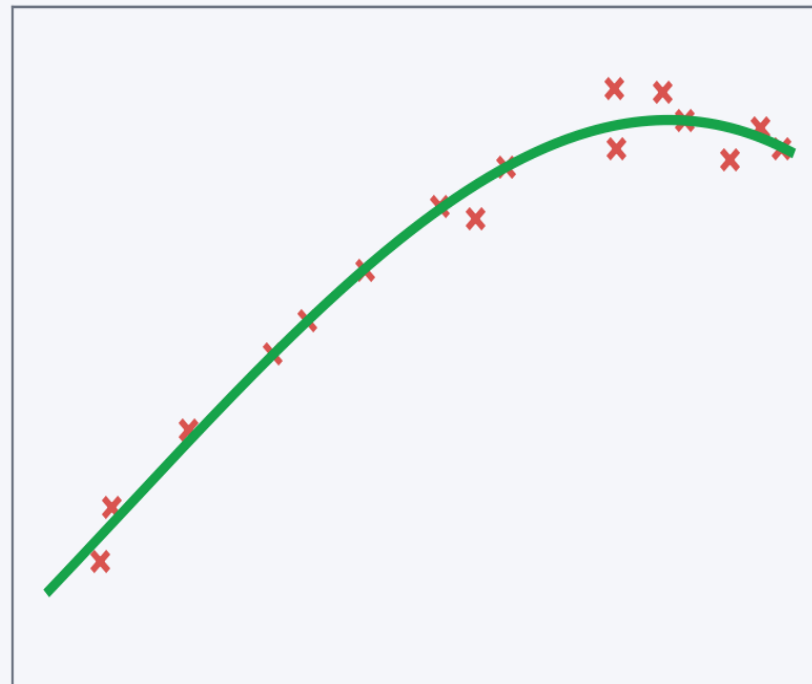
Ridge · Lasso · Elastic Net

Trực giác Regularization

Trước: θ_3, θ_4 lớn $\rightarrow$ quá khớp



Sau: phạt $\theta_3, \theta_4 \approx 0 \rightarrow$ mượt



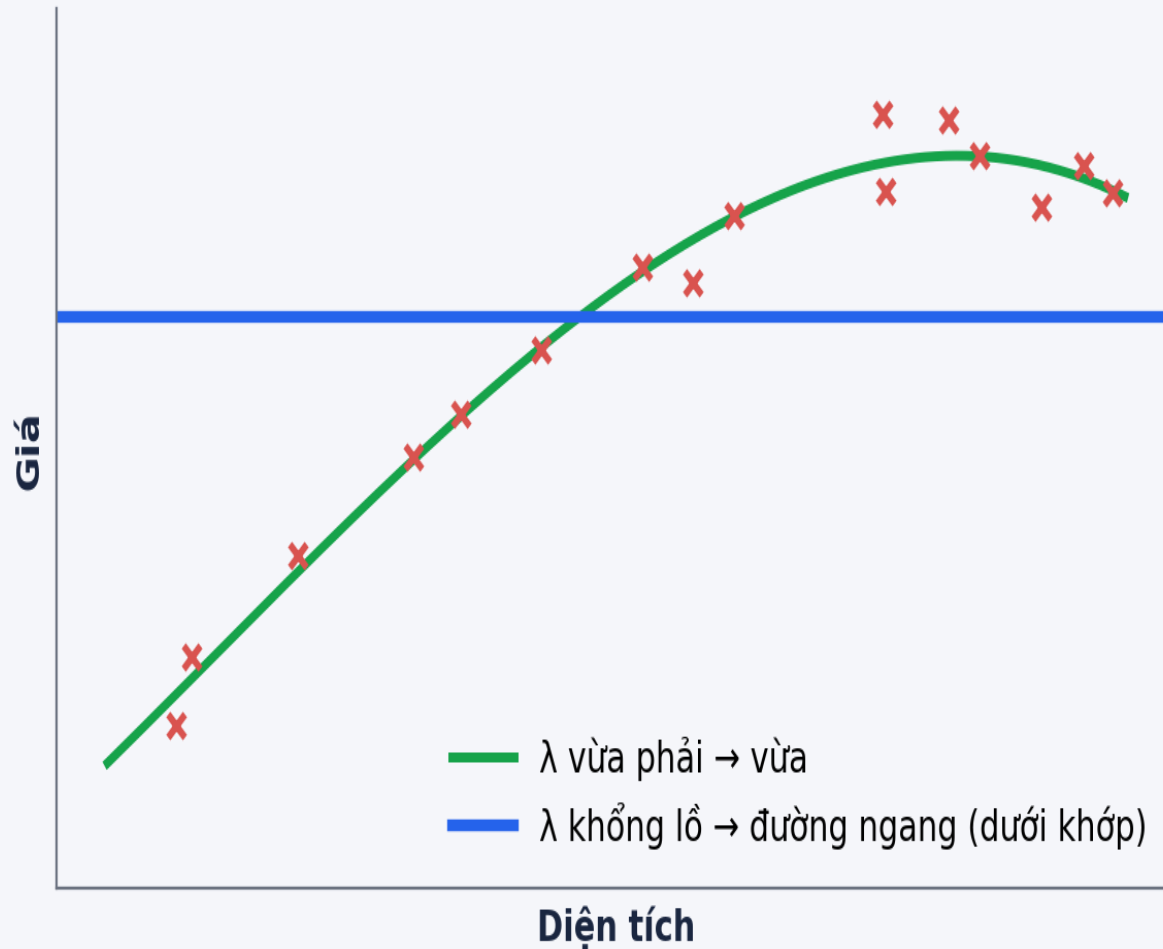
- **Ý tưởng:** thêm phạt để ép các hệ số bậc cao ($\theta_3, \theta_4 \dots$) ≈ 0 — ví dụ thêm $+1000 \cdot \theta_3^2 + 1000 \cdot \theta_4^2$ vào hàm chi phí.
- *Hệ số nhỏ $\rightarrow$ đường cong mượt hơn $\rightarrow$ giả thuyết “đơn giản” hơn $\rightarrow$ bớt quá khớp.*

Ridge (L2): Hàm chi phí

$$J(\theta) = \frac{1}{2m} \left[\sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 + \lambda \sum_{j=1}^n \theta_j^2 \right]$$

- **λ (lambda):** tham số regularization — điều khiển mức phạt.
- **θ_0 KHÔNG bị phạt** (chỉ phạt $\theta_1 \dots \theta_n$).
- θ nhỏ $\Rightarrow$ giả thuyết đơn giản hơn $\Rightarrow$ ít quá khớp. L2 = phạt tổng bình phương hệ số.

Ảnh hưởng của λ



- **λ quá lớn** (vd 10^{10}): mọi $\theta_j \approx 0 \rightarrow$ mô hình thành đường ngang $\rightarrow$ **dưới khớp** (high bias).
- **λ quá nhỏ**: phạt yếu $\rightarrow$ vẫn quá khớp.
- **Chọn λ bằng cross-validation** (RidgeCV) để cân bằng bias–variance.

Gradient Descent + Weight Decay

- Bước cập nhật cho θ_j ($j \geq 1$) khi có regularization L2:

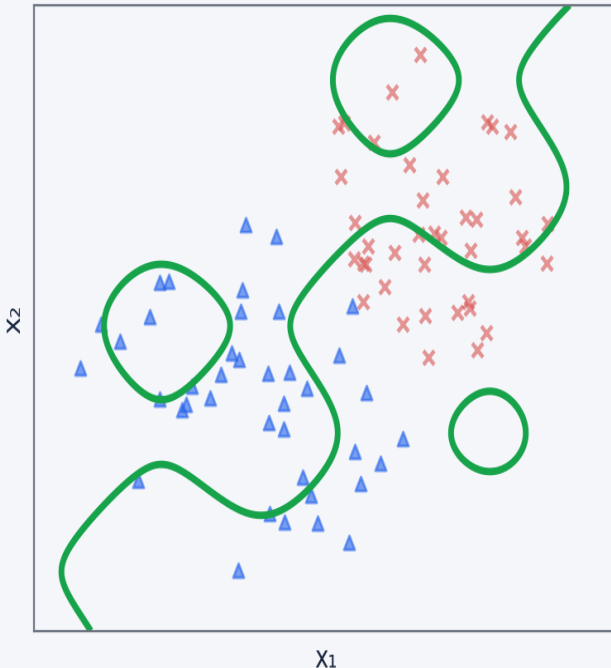
$$\theta_j := \theta_j \left(1 - \alpha \frac{\lambda}{m}\right) - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

- **Hệ số $(1 - \alpha\lambda/m) < 1$** = weight decay: mỗi bước CO NHỎ trọng số một chút trước khi cập nhật theo gradient.
- **So với bản không regularization:** chỉ thêm thừa số co $(1 - \alpha\lambda/m)$; θ_0 vẫn cập nhật như cũ.

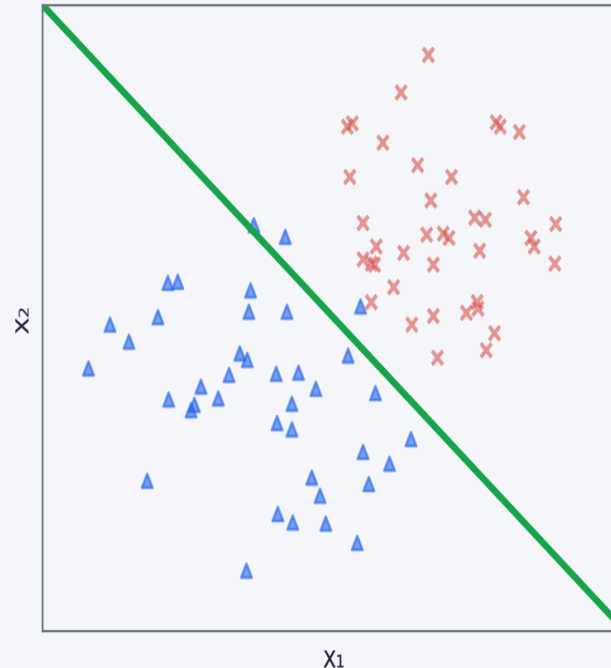
Regularized Logistic Regression

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log h_{\theta}(x^{(i)}) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)})) \right] + \frac{\lambda}{2m} \sum_{j=1}^n \theta_j^2$$

Không regularization → ranh giới ngoằn ngoèo



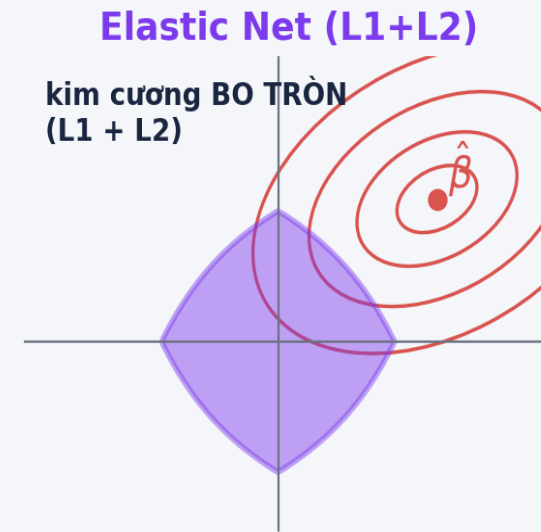
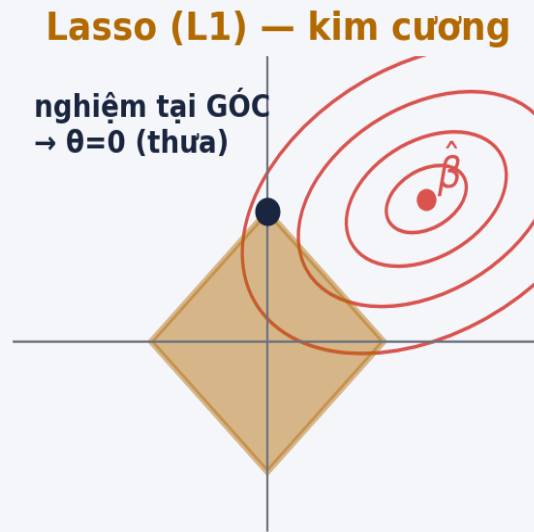
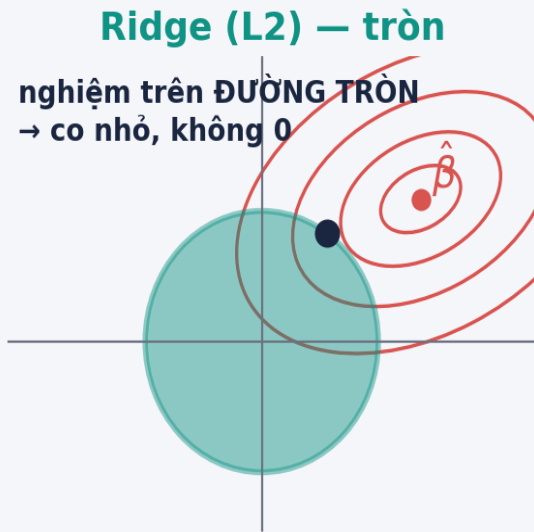
Có regularization → ranh giới mượt



- Cùng dạng phạt $\lambda/2m \cdot \sum \theta_j^2$ thêm vào hàm chi phí cross-entropy.
- **Tác dụng:** ranh giới quyết định MƯỢT hơn, bớt ngoằn ngoèo → tổng quát hóa tốt hơn.
- Quy tắc cập nhật GD giống Ridge, với h là sigmoid.

Lasso (L1) & hình học L1 / L2

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 + \lambda \sum_{j=1}^n |\theta_j|$$



LASSO = Least Absolute Shrinkage and Selection Operator. L1 đẩy nhiều hệ số về ĐÚNG 0 → CHỌN BIẾN tự động; L2 chỉ co nhỏ chứ không về 0.

Elastic Net (L1 + L2)

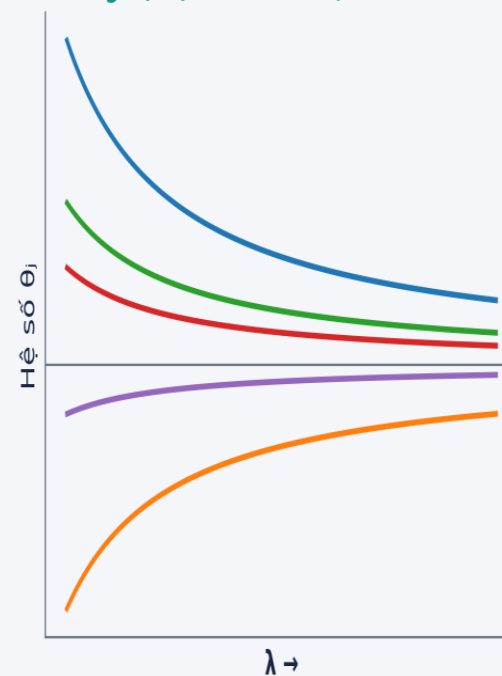
$$J(\theta) = \text{MSE} + \lambda \left[\alpha \sum_{j=1}^n |\theta_j| + (1 - \alpha) \sum_{j=1}^n \theta_j^2 \right]$$

- **Kết hợp ưu điểm:** L1 cho tính THỪA (chọn biến) + L2 cho tính ỔN ĐỊNH (xử lý đa cộng tuyến, nhóm biến tương quan).
- **α điều phối tỷ trọng L1/L2:** $\alpha=1 \rightarrow$ Lasso thuần; $\alpha=0 \rightarrow$ Ridge thuần.
- *Phù hợp khi có nhiều đặc trưng tương quan theo nhóm — Lasso thuần dễ chọn lệch 1 biến tùy ý.*

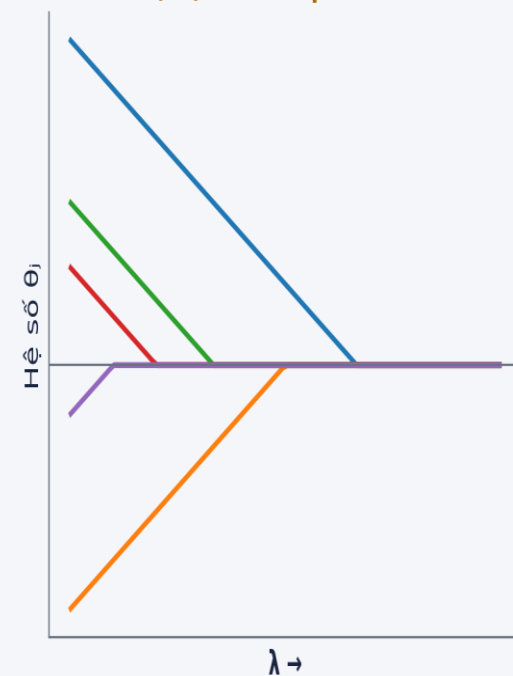
So sánh Ridge / Lasso / Elastic Net

Tiêu chí	Ridge (L2)	Lasso (L1)	Elastic Net
Phạt	$\sum \theta_j^2$	$\sum \theta_j $	$\alpha \cdot L1 + (1-\alpha) \cdot L2$
Đưa hệ số về 0?	Không	Có (thừa)	Có
Chọn biến	Không	Có	Có
Đa cộng tuyến	Tốt	Kém (chọn 1 tùy ý)	Tốt
Nghiệm	Dạng đóng	Lặp	Lặp

Ridge (L2): co nhỏ dần, KHÔNG về 0



Lasso (L1): nhiều hệ số về ĐÚNG 0



Đường co hệ số theo λ : Lasso đưa về 0, Ridge chỉ co nhỏ

04

Hướng 2: Chọn đặc trưng

Feature Selection

Tổng quan: Filter / Wrapper / Embedded

FILTER

- Đánh giá đặc trưng ĐỘC LẬP với mô hình, dựa trên thống kê.
- **Nhanh – rẻ**
- **Ví dụ:** Variance, Correlation, χ^2 , Mutual Information

WRAPPER

- Dùng HIỆU NĂNG của mô hình để chọn tập đặc trưng tốt nhất.
- **Chính xác – tốn thời gian**
- **Ví dụ:** Forward/Backward, RFE, Boruta, mRMR

EMBEDDED

- Chọn đặc trưng diễn ra NGAY trong quá trình huấn luyện.
- **Cân bằng**
- **Ví dụ:** LASSO (L1), Random Forest importance

Filter — Missing & Low Variance

- **Missing Value Ratio:** thiếu quá nhiều → ít thông tin → loại theo ngưỡng.
- **Low Variance Filter:** đặc trưng gần như hằng → vô dụng để phân biệt mẫu → loại theo ngưỡng phương sai.
- ⚠ **Lưu ý:** chuẩn hóa TRƯỚC, vì phương sai phụ thuộc thang đo.

```
from sklearn.feature_selection \
    import VarianceThreshold

# loại đặc trưng gần như hằng
sel = VarianceThreshold(
    threshold=0.01)
X_new = sel.fit_transform(X)
```

Filter — High Correlation

- **Cặp biến tương quan cao** → thông tin trùng lặp → giữ lại MỘT.
- Chọn kiểm định theo kiểu cặp biến:

Kiểu cặp biến	Kiểm định phù hợp
Số ↔ Số	Pearson (tuyến tính) / Spearman (đơn điệu)
Phân loại ↔ Phân loại	Chi-square (χ^2)
Số ↔ Phân loại	ANOVA F-test

```
from sklearn
    .feature_selection import \
        SelectKBest, chi2
X_new = SelectKBest(
    chi2, k=20).fit_transform(
        X, y)
```

Filter — Mutual Information

- **Đo lượng thông tin chia sẻ** giữa một đặc trưng và biến mục tiêu y .
- **Ưu điểm lớn:** bắt được quan hệ PHI TUYẾN (correlation chỉ bắt quan hệ tuyến tính).
- **Ví dụ $y = x^2$:** Pearson ≈ 0 nhưng $MI > 0$.
- $MI \geq 0$; $MI = 0 \Leftrightarrow$ đặc trưng và y độc lập. MI cao $\Rightarrow$ ưu tiên giữ.

```
from sklearn.feature_selection \
    import mutual_info_classif,
           SelectKBest

# classif: phân loại · regression: hồi
# quy
X_new = SelectKBest(
    mutual_info_classif, k=20)
    .fit_transform(X, y)
```

Wrapper — Forward & Backward Selection

Forward Selection

- Bắt đầu RỖNG; thử từng biến đơn lẻ.
- Thêm dần biến làm TĂNG hiệu năng nhiều nhất.
- Lặp đến khi không còn cải thiện đáng kể.

Backward Elimination

- Bắt đầu với TOÀN BỘ n biến.
- Mỗi lần bỏ thử một biến, đo hiệu năng.
- Bỏ hẳn biến mà việc loại ít/không làm giảm hiệu năng. Lặp đến khi không bỏ thêm được.

⚠️ Cả hai đều TỐN THỜI GIAN — phải huấn luyện mô hình rất nhiều lần.

Wrapper — RFE / RFECV, Boruta, mRMR

- **RFE**: lặp loại đệ quy các đặc trưng yếu nhất theo trọng số mô hình.
- **RFECV**: tự chọn SỐ đặc trưng tối ưu bằng cross-validation.
- **Boruta**: so importance đặc trưng thật với “shadow features” (bản xáo trộn).
- **mRMR**: Max-Relevance, Min-Redundancy — tránh chọn biến trùng nhau.

```
from sklearn.feature_selection \
    import RFE, RFECV

sel = RFE(est, n_features_to_select=5)
sel.fit(X, y)
sel.support_    # mặt nạ giữ/loại
sel.ranking_    # 1 = được chọn

# tự chọn số đặc trưng bằng CV
RFECV(est, step=1, cv=5).fit(X, y)
```

Embedded — LASSO (L1)

- **Cầu nối với Mục 3:** phạt $L1 = \alpha \cdot \sum |\text{hệ số}|$ đẩy nhiều hệ số về ĐÚNG 0.
- Hệ số = 0 $\Rightarrow$ đặc trưng tương ứng bị loại TỰ ĐỘNG.
- **α lớn** $\Rightarrow$ phạt mạnh $\Rightarrow$ giữ ít đặc trưng (mô hình thưa).
- **Mẹo:** LassoCV tự dò α tốt nhất bằng cross-validation.

```
from sklearn.linear_model import Lasso
from sklearn.feature_selection \
    import SelectFromModel

las = Lasso(alpha=0.01).fit(X, y)
sel = SelectFromModel(las, prefit=True)
X_new = sel.transform(X)
# hệ số = 0 -> biến bị loại
```

Embedded — RF Importance & Permutation

- **RF feature_importances_**: theo mức giảm impurity khi tách — nhanh, có sẵn sau khi train.
- ⚠️ **Thiên vị** đặc trưng nhiều giá trị (high-cardinality).
- **Permutation Importance**: xáo trộn 1 cột, đo mức GIẢM hiệu năng — không thiên vị, dùng cho MỌI mô hình, đo trên test.
- **Cặp đôi đề xuất**: RF (nhanh, khám phá) + Permutation (đáng tin, kết luận); SHAP để giải thích từng dự đoán.

```
from sklearn.inspection \
    import permutation_importance

r = permutation_importance(
    model, X_test, y_test,
    n_repeats=10, random_state=0)
r.importances_mean # độ quan trọng
r.importances_std  # độ ổn định
```

Tóm tắt

- **Tiền xử lý là bắt buộc:** xử lý dữ liệu thiếu (xóa / điền / cờ), chuẩn hóa (z-score, min-max, robust), mã hóa biến phân loại đúng kiểu (one-hot / ordinal / target).
- **Overfitting** = khớp train tốt nhưng tổng quát kém (variance cao) → cân bằng bias–variance.
- **Hướng 1 — Regularization:** Ridge (L2) co nhỏ hệ số; Lasso (L1) đưa về 0 & chọn biến; Elastic Net kết hợp cả hai.
- **Hướng 2 — Chọn đặc trưng:** Filter (thống kê, nhanh), Wrapper (theo hiệu năng mô hình), Embedded (Lasso, RF importance).